Plan — Web de ropa urbana (Fase 1: catálogo + venta por WhatsApp)

---

1. Objetivo

Web de ropa underwear y urbana con identidad visual muy fuerte. En esta fase la web funciona como **catálogo completo sin pasarela de pago**: la venta se concreta por WhatsApp. El admin gestiona productos, variantes, stock y drops desde un panel.

2. Stack

Capa	Tecnología	Nota
Frontend	Next.js (App Router) + Tailwind CSS + Framer Motion	Animaciones = personalidad urbana
Backend	Laravel como API REST JSON	Auth, migraciones y ORM listos
Admin	Filament sobre Laravel	Admin funcional en días, no semanas
Base de datos	MySQL	
Carrito	Zustand + localStorage	Sin backend, sin cuentas de usuario en esta fase
Deploy	Vercel (frontend) + host PHP/MySQL o VPS (backend)	Dos deploys: resolver CORS en Fase 0

3. Documentos complementarios

- `schema.sql` — esquema de base de datos de referencia (qué deben producir las migraciones de Laravel)
- `api-contracts.md` — contratos de la API: endpoints, JSON exacto, errores

4. Modelo de datos (resumen)

```
users            → equipo admin (roles: admin / editor)
categories       → Camisetas, Hoodies, Underwear...
collections      → drops ("published_at" futuro = próximamente)
products         → el producto padre (precio aquí, stock NO)
product_images   → galería ordenada
variants         → color + talla + stock   ← LA PIEZA CLAVE
settings         → whatsapp_number, redes, etc. (clave/valor)
```

Decisiones clave:

- **El stock vive en `variants`**, no en `products`: un hoodie negro en M y en L son dos filas distintas.

- **El carrito no toca la BD** en esta fase (localStorage).
- Precios en `DECIMAL(10,2)`, nunca FLOAT.

5. Flujo de venta

1. Usuario agrega productos al carrito (variante concreta: color + talla)
2. Página de carrito → botón **"Confirmar pedido por WhatsApp"**
3. Se genera el mensaje automáticamente (productos + tallas + total) y abre `wa.me/<número>?text=...`
4. La venta se cierra en el chat

Decisión asumida: sin reserva de stock en esta fase. Si dos personas piden lo mismo, el admin lo responde en el chat. Es lo normal en esta etapa.

6. Fases

Fase 0 — Fundamentos

- ☐ **B**: Proyecto Laravel + Filament, migraciones y modelos según `schema.sql`, seeders, CORS configurado, API skeleton
- ☐ **A**: Proyecto Next.js, Tailwind con tokens de la identidad de marca, layout base, tipografías, fuentes cargadas
- ☐ **C**: Repo único (`/frontend` + `/backend`), README con convenciones, deploy de staging de ambos
- ☐ **Todos**: revisar y acordar `api-contracts.md`

Fase 1 — Catálogo público (el corazón visual)

- ☐ **A**: Home (hero, destacados, último drop), grid con filtros, animaciones y transiciones
- ☐ **B**: Endpoints públicos según contrato, manejo de imágenes (storage + URLs), seed con productos reales
- ☐ **C**: Ficha de producto — galería, selector de color/talla, estados de stock (talla agotada tachada, "próximamente")

Fase 2 — Venta por WhatsApp

- ☐ **A**: Carrito (Zustand + localStorage), drawer del carrito, badge de contador
- ☐ **C**: Página de carrito + generador del mensaje (productos + variantes + total → `wa.me`)
- ☐ **B**: Endpoint de settings públicos (número de WhatsApp configurable)

Fase 3 — Admin

- ☐ **B + C**: Filament — CRUD de productos con relation managers (variantes, imágenes), categorías, colecciones, gestión rápida de stock

- ☐ **B**: Auth, roles admin/editor (Filament Shield), página de settings

Fase 4 — Pulido y lanzamiento

- ☐ SEO: metadata dinámica, sitemap.xml, robots.txt, JSON-LD de producto
- ☐ Analytics: GA4 + Meta Pixel
- ☐ Performance: Next/Image, caché, lazy loading
- ☐ **A**: Páginas de marca — lookbook, about (en streetwear esto vende)
- ☐ Deploy a producción, dominio, HTTPS

Duración relativa: Fase 1 y 3 son las más pesadas (~2 semanas cada una asumiendo dedicación similar); Fases 0, 2 y 4 ~1 semana cada una.

7. División de roles

Rol	Responsabilidad principal
A — Frontend/Estilo	Design system, home, grid, animaciones, lookbook
B — Backend/API	BD, API REST, auth, imágenes, settings
C — Fullstack/Admin	Panel admin, carrito + WhatsApp, QA, deploys

8. Riesgos y mitigación

1. **CORS y URL de la API** entre Vercel y el host PHP → resolverlo en Fase 0 con un endpoint `GET /api/v1/ping` de prueba
2. **Imágenes pesadas** → WebP obligatorio, tamaño máximo al subir, empezar con storage de Laravel público; S3/R2 solo si crece
3. **Tiempos** → si alguien es part-time, Fases 1 y 3 se alargan proporcionalmente

9. Estructura del repo

```
/tienda-urbana
  /frontend     → Next.js (deploy: Vercel)
  /backend      → Laravel + Filament (deploy: host PHP / VPS)
  README.md     → cómo levantar cada uno, convenciones, enlaces a este plan
```